SemLake: Predicate-Level Three-Way Merge for Branched Data Lakehouses

Xiaofan Cao
xc2314895@qu.edu.qa

ABSTRACT

Lakehouse catalogs such as lakeFS, Nessie and Bauplan let a pipeline fork a table in milliseconds, but merging two branches back is decided by physical overlap: which files, or which rows, both branches wrote. This rule is wrong in both directions. Two branches that update different rows of the same Parquet file conflict under file-level optimistic concurrency control, and a compliance purge on one branch merges silently with a report on the other that aggregated the purged rows, because the two branches wrote disjoint files. We present SemLake, a three-way merge for branched lakehouse tables that decides conflicts on what each branch read and deleted. Each commit stores a small summary with its row changes, the pre-images of deleted rows, and the `WHERE` clauses of its reads and deletes. A merge replays only the summaries written since the fork, tests read and delete clauses for overlap with a static satisfiability check over a conjunctive fragment, and falls back to evaluating the read clause on the deleted rows' recorded values. Rows changed by both branches are combined by column; true cell collisions are kept as explicit choices. The merged snapshot is written as deletion-vector updates and one delta file, so no base file is rewritten. On 27 labelled merge scenarios, SemLake reaches recall 1.00 and precision 1.00; Dolt reaches recall 0.14, and a model of Delta Lake's row-level checks reaches precision 0.65. On 42 refresh-versus-query pairs over real TPC-H `lineitem`, SemLake reaches recall 1.00 and precision 1.00. A merge on a 10K-file table issues 7 object-store requests, against 20,007 for copy-on-write. In a simulated swarm of up to 64 agents committing to one table, SemLake committed no hazards over 5 seeds, against 63 for file-level concurrency control.

PVLDB Reference Format:
Xiaofan Cao. SemLake: Predicate-Level Three-Way Merge for Branched Data Lakehouses. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/WanderingSnail/semlake_milestone2.

1 INTRODUCTION

Lakehouse tables keep their data in immutable Parquet files on object storage and describe each version of the table with a snapshot of metadata [4, 6, 33]. Because a snapshot is only metadata, a table can be branched by copying a pointer. lakeFS [31] and Nessie [26] use this to give data pipelines Git-like branches: a write-audit-publish job, a backfill or a regulatory purge runs on its own branch and is merged into `main` when it is done. Agent frameworks are starting to do the same for AI agents that work on shared data [17, 34].

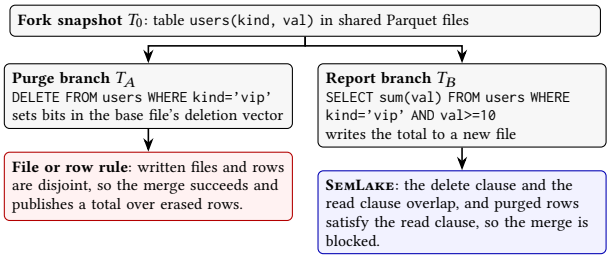


Figure 1: A purge and a report that touch disjoint files and rows. Physical merge rules accept the pair; SemLake rejects it from commit metadata.

Branching is cheap; merging is where the semantics live, and today it is decided by physical overlap. Iceberg and Delta Lake reject a commit when a concurrent commit changed a file it also changed [4, 6]. Delta Lake's row-level concurrency narrows write conflicts to rows, but still rejects a transaction when a concurrent commit deleted rows from a file it read [22]. Version-controlled databases such as Dolt merge at cell granularity and check declared constraints [12]. None of these rules looks at what a branch actually depended on, and each fails in one direction or the other (Figure 1):

- *False conflicts.* Two branches that update different rows of the same 128 MB file conflict under file-level concurrency control. A row-level variant avoids the write conflict, but a report that scanned the file is still rejected when the other branch deletes one unrelated row from it.
- *Silent corruption.* A purge branch deletes the VIP customers while a report branch computes revenue over the VIP customers and writes it to a new file. The two branches wrote disjoint files and disjoint rows, so every physical rule merges them. The merged table now publishes a figure derived from rows it claims to have erased.

Program-merge research has long treated three-way merge as a semantic question, with verified merges [28] and explicit representations of conflicting candidates [35]. Those techniques reason about programs. A lakehouse merge must instead reason about which rows two branches read and removed, without scanning data files, because each object-store request costs tens of milliseconds and is billed [3, 13].

SemLake is a three-way merge for branched lakehouse tables built on that idea. Each commit writes a compact summary: its row changes, the values each deleted row had, and the `WHERE` clauses of

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

the statements that read or deleted rows. A merge finds the fork point, replays the summaries written since then on each side, and checks every delete on one side against every read on the other. A static satisfiability test over a conjunctive fragment rules out most pairs at once; for the rest, the read clause is evaluated on the recorded values of the deleted rows. Rows changed by both sides are combined column by column, and a cell both sides set to different values is kept as an explicit choice. The merged snapshot is published with deletion vectors and a single delta file, so the merge touches only metadata and the changed rows.

This paper makes four contributions:

(1) A merge model for branched tables with four criteria, one per workload class, and the *purge-safety* invariant for reads and deletes. We show that purge-safety and the serializability that SSI enforces are incomparable, and that purge-safety is the stricter of the two on read–delete pairs (Section 3).
(2) A conflict check that combines a static clause test with a check of the deleted rows' recorded values, and a replay of commit summaries since the fork that stays correct across chains of earlier merges. We prove that a merge the check accepts is purge-safe (Sections 4 and 5).
(3) A merge that publishes deletion-vector updates and one delta file, with request counts that depend on the branch delta rather than on the table size (Section 4.5).
(4) A prototype and an evaluation against file-level concurrency control, a model of Delta Lake's row-level checks, copy-on-write, tuple versioning, a CRDT merge, Dolt and an SSI model, on labelled scenarios, real TPC-H data and a simulated agent swarm (Sections 6 and 7).

2 BACKGROUND AND PROBLEM

2.1 Lakehouse Tables and Deletion Vectors

A lakehouse table is a set of immutable Parquet files plus a metadata log; each commit produces a snapshot that lists the live files [4, 6]. Files are large: Iceberg's default target is 512 MB [5]. Updating one row by copy-on-write therefore rewrites a whole file. Merge-on-read avoids this: a *deletion vector* (DV), a Roaring bitmap [20] attached to a data file, marks the positions of deleted or superseded rows, and new row versions go to a small delta file [4, 21, 25]. Readers skip masked positions, and compaction later folds the DVs back into clean files. Object stores charge per request and add tens of milliseconds per round trip [3, 13], so the number of requests a merge issues matters as much as its CPU time.

2.2 Workloads and Metrics

Two branches T_A and T_B fork from a snapshot T_0 and are merged into T_M. We use four workload classes, each named by what the two branches do:

W1, shared-file updates. The branches update or delete different rows that live in the same files.
W2, hot rows. The branches update overlapping rows; a fraction κ of the changed rows is shared.
W3, read–delete dependencies. One branch deletes rows that the other branch read to compute a result, or that it requires to stay live (a foreign-key parent, a row it cites), or it writes values that break a value rule the other branch declared.
W4, schema and scale. One branch changes the schema while the other appends data; the table has many files and a deep commit history.

We measure four properties:

M1, correctness. Recall and precision of conflict detection on W3 and W4, and the false-abort rate on W1.
M2, merge latency. CPU time of the merge, and the end-to-end time once object-store round trips are added, as the table, the history and κ grow.
M3, object-store cost. Requests and bytes written per merge, and dollars per 1,000 merges.
M4, read overhead. The extra time of an analytical scan over a merged snapshot whose DVs have not been compacted.

Section 7.6 adds a fifth workload, a swarm of agents that branch from and commit to one table, and reports commit throughput for it.

2.3 Existing Merge Rules

Table 1 lists the rules we compare against. B1, B1R, B2 and B6 are decision rules that we implement over the same snapshots as SemLake; B3 and B4 are merge strategies from the dataset-versioning and replicated-data literature that we reimplement; B5 is the Dolt binary.

B1, file-level OCC. The default of Iceberg and Delta Lake [4, 6]: two commits conflict if they changed a common file.
B1R, Delta-style row-level checks. Same-row writes conflict; a commit also conflicts when a concurrent commit deleted or changed rows in a file it read (`ConcurrentDeleteReadException` [22]). We model B1R in its best case for data skipping, in which a branch reads only the files that contain at least one row matching its filter.
B2, row-level copy-on-write. Rows are merged by identifier, as in copy-on-write tables, and every changed file is rewritten.
B3, tuple versioning. Per-file version dictionaries in the style of Decibel and OrpheusDB [18, 23].
B4, CRDT merge. Last-writer-wins per cell, as in conflict-free replicated views [2, 15].
B5, Dolt. Cell-level merge with declared `FOREIGN KEY` and `CHECK` constraints [12].
B6, SSI. Each branch is one transaction on T_0; the merge fails only if read–write dependencies run in both directions, as in serializable snapshot isolation [9], and same-row writes fail.

3 MERGE MODEL

3.1 Snapshots and Branch Deltas

Let $\mathcal{R}$ be a set of stable row identifiers, as in Iceberg's row lineage [4], and $\mathcal{A}$ a set of column names.

Definition 3.1 (Snapshot). A snapshot is $T = (id, P, S, rows, \Phi)$: an identifier, a tuple P of zero, one or two parent identifiers, a schema S, a partial map $rows : \mathcal{R} \rightharpoonup (\mathcal{A} \rightharpoonup \mathcal{V})$ from live row identifiers to

Table 1: What each merge rule compares, and how it treats each workload class. "Merges" means the pair is accepted without a check.

Rule	Compares	W1	W3
B1 file OCC	written files	aborts	merges
B1R Delta-style	written rows, read files	merges	blocks any read file
B2 copy-on-write	written rows	rewrites files	merges
B3 tuple versioning	written rows	merges	merges
B4 CRDT	cells, last writer wins	merges	merges
B5 Dolt	cells, declared constraints	merges	declared FK/CHECK only
B6 SSI	read and write sets	merges	two-way cycles only
SemLake	rows, read/delete clauses	merges	clause and row check

Table 2: Predicate kinds and the conflict each one raises when branch X's changes meet branch Y's predicate.

Kind on Y	Example	Conflict when X…
`reads_where`(φ_R)	`kind='vip' AND val>=10`	deleted a row that satisfied φ_R (Def. 3.3)
`deletes_where`(φ_D)	`l_shipdate < 19930201`	(scope of Y's own deletes; checked against X's reads)
`rows_live`(I)	$I = \{101, 105\}$	deleted a row in I
`foreign_key`(I)	parents of new orders	deleted a row in I
`domain`(φ)	`val >= 0`	inserted or updated a value outside φ

values, and a map Φ from file identifiers to file descriptors (object key, size, deletion vector).

Snapshots and parent edges form a DAG. The generation of a snapshot is 0 without parents and one more than its highest parent otherwise; $\text{anc}(u)$ is the set of ancestors of u, including u. The fork point of two tips is $T_0 = \text{LCA}(T_A, T_B)$, the common ancestor of highest generation, with ties broken by identifier.

Definition 3.2 (Commit summary and branch delta). A commit c with one parent records a summary $(ops_c, S_c, \Pi_c, \sigma_c)$: the operation on each changed row (insert, update with its assignments, or delete), the schema after c, the predicates attached to c, and for each row c deletes its values just before the delete. Folding the commits in $\text{anc}(T_B) \setminus \text{anc}(T_0)$ in topological order gives the *branch delta* Δ_B with operations ops_B, deleted rows $D(B)$, delete-time values σ_B, tip schema S_B and predicates $\Pi(B)$.

Table 2 lists the five predicate kinds. Read and delete clauses are extracted from SQL `SELECT` and `DELETE` statements with SQLGlot; the other kinds are attached by the writer.

3.2 Merge Criteria

A merge $T_M = \text{merge}(T_0, T_A, T_B)$ must satisfy four criteria, one per workload class.

Criterion 1 (W1, convergence). A row changed by one branch, or changed identically by both, takes that change. A row both branches updated on disjoint columns takes both updates.

Criterion 2 (W2, explicit collisions). If both branches set the same cell to different values, or one updates a row the other deletes, the cell is recorded as a choice `choice`$(r, c, [(A, v_A), (B, v_B)])$ in a choice set C_M, in the style of version-space merges [35]. The other cells of the row still converge, and the contested cell keeps its value from T_0 until a later commit resolves the choice.

Criterion 3 (W3, predicates). The merge is purge-safe (Definition 3.3), no row listed in a partner's `rows_live` or `foreign_key` predicate is deleted, and every value a branch assigns satisfies the partner's `domain` predicates. Unlike a `CHECK` constraint, a `domain` predicate constrains only the values assigned since T_0, so legacy rows that violate it do not block the merge.

Criterion 4 (W4, schemas). Neither branch drops or renames a column of S_0; every added column is nullable; and each column's type on each branch is τ_0 or a widening of it. The prototype allows `int32`→`int64`, `float`→`double` and integer→`double`. The last of these can lose precision above 2^{53}; Iceberg allows only the first two [4], and a deployment should use Iceberg's list. S_M takes the wider type of each column without rewriting files.

A merge that meets Criteria 3 and 4 returns T_M with status `clean` if C_M is empty and `choice` otherwise. A merge that fails Criterion 3 returns `semantic_conflict`, and one that fails Criterion 4 returns `schema_conflict`; neither writes anything.

3.3 Purge-Safety

A report on Y read the rows of T_0 that satisfied its clause. If X deleted one of them, T_M contains both the report's result and the deletion. We treat a row as relevant to the report if it satisfied the clause at the fork, which is what the report saw, or when X deleted it, which covers a row that X first moved into the report's range.

Definition 3.3 (Purge-safe merge). A merge of T_A and T_B with fork T_0 is *purge-safe* if, for $(X, Y) \in \{(A, B), (B, A)\}$, every `reads_where`$(\varphi_R) \in \Pi(Y)$ and every row $r \in D(X)$: $T_0.rows(r) \not\models \varphi_R$ (when r exists in T_0) and $\sigma_X(r) \not\models \varphi_R$.

The definition is about data, not about how a system tests it. Two limits are deliberate. It covers deletes but not updates: an update on X that moves a row out of φ_R also changes what Y's report would compute, and the prototype does not flag it (Section 9). And a row that satisfied φ_R only at delete time counts as a conflict even though the report never saw it; this is conservative.

Proposition 3.4. *Purge-safety and two-branch serializability are incomparable. In particular, some purge-safe merges are not serializable, and some merges that SSI accepts are not purge-safe.*

Proof. Not purge-safe but accepted by SSI: take Figure 1. T_B reads the VIP rows and writes a new row; T_A deletes the VIP rows and reads nothing T_B writes. The dependency graph [1] has one anti-dependency, $T_B \xrightarrow{rw} T_A$, and no edge back, so it has no cycle and the order T_B before T_A is serializable; SSI accepts it [9]. A deleted VIP row satisfies φ_R, so the merge is not purge-safe.

Purge-safe but not serializable: let both branches update the same cell of the same row to different values, with no predicates. Definition 3.3 holds trivially. Any serial order loses one of the two writes, so a merge that keeps both outcomes as a choice is not equivalent to a serial execution; this is why Criterion 2 handles such pairs separately. □

The proof shows where purge-safety is stricter: on a read–delete pair with a dependency in one direction only. In a branched lakehouse both outputs are published together in T_M, so the order SSI picks does not exist for a reader of T_M: the table claims the rows are gone and publishes a figure computed from them.

3.4 The Static Clause Fragment

The fragment $\mathcal{F}$ contains conjunctions of atoms $c\ op\ \ell$ with $op \in \{=, \neq, <, \leq, >, \geq\}$, c `IN` $(\ell_1, \ldots, \ell_k)$ and c `BETWEEN` ℓ_1 `AND` ℓ_2, over column names c and literals ℓ. Satisfiability of a conjunction in $\mathcal{F}$ is decided column by column: collect the allowed values from = and `IN`, the excluded values from ≠, and the tightest lower and upper bounds, then check whether any value remains. Clauses with `OR`, `NOT`, arithmetic, column comparisons, joins or subqueries are outside $\mathcal{F}$.

4 DESIGN

SemLake has three parts. The *catalog* keeps branch heads, which it advances by compare-and-swap, and the commit DAG. The *commit layer* writes a data file or DV change and one JSON summary per commit. The *merge engine* replays summaries since the fork (Algorithm 1), checks predicates (Algorithm 2), reconciles rows, and publishes the merged snapshot.

4.1 Commit Summaries

Each commit writes `manifests/<id>.summary.json` with its parents, branch and schema; the descriptors of files whose DV or contents changed; the operation and assignments for each changed row, and the values of each deleted row; its predicates; and any open choices. Finding what a branch changed since the fork therefore costs one `GET` per commit and no data-file read, regardless of how many files the table has. Writers can attach predicates directly or pass SQL text; for a `SELECT` or `DELETE`, the extractor normalizes the `WHERE` clause and records a `reads_where` or `deletes_where` predicate. A clause with a join or subquery is kept as text and handled conservatively.

4.2 Replay Since the Fork

Long-lived branches merge repeatedly, so the history between the fork and a tip contains earlier merge commits. Reading only each tip's summary is wrong: a merge commit's summary does not repeat the predicates of the commits it merged, so a later merge would forget an earlier read. Walking every ancestor is wrong too: it replays history that the fork already contains, which looks like concurrent changes.

Algorithm 1 walks back from the tip, stops at the fork's ancestors, and folds the commits it finds in topological order. Fold applies a commit's row operations to the running delta; when a row is deleted, it records the row's values at that point in σ, and when a row with an open choice is updated, the choice is resolved. At an earlier merge u, the walk strips the part both parents share ($\ominus$) and combines the rest, so predicates and choices that reached u from either parent are kept without a second read of their summaries.

4.3 Conflict Check

Algorithm 2 applies Criterion 3 in both directions.

Algorithm 1 ReplaySinceFork(b, t): the delta of tip t since fork b.

```
1:  if t = b then return the empty delta
2:  skip ← anc(b)                                          ▷ cached ancestor set
3:  folded[b] ← empty delta
4:  for u in a post-order walk from t that stops at skip do
5:    if u has parents p1, p2 then                          ▷ an earlier merge
6:      f ← LCA(p1, p2); if f ∉ folded then f ← b
7:      L ← folded[p1] ⊖ folded[f];  R ← folded[p2] ⊖ folded[f]
8:      folded[u] ← Fold(folded[f], Combine(L, R))
9:    else                                                  ▷ one summary GET
10:     p ← parent of u, or b if it lies in skip
11:     folded[u] ← Fold(folded[p], Summary(u))
12: return folded[t]
```

Algorithm 2 CheckPredicates$(\Delta_A, \Delta_B, T_0)$: the list of violations.

```
1:  V ← [ ]
2:  for (X, Y) ∈ {(A, B), (B, A)} do
3:    moved ← {r ∈ D(X) : σ_X(r) ≠ T0.rows(r)}
4:    for p ∈ Π(Y) of kind rows_live or foreign_key do
5:      if p.I ∩ D(X) ≠ ∅ then add a violation
6:    for s ∈ Π(X) of kind deletes_where, p ∈ Π(Y) of kind reads_where do
7:      if ¬Overlap(s.φ, p.φ) then                          ▷ static test over F
8:        if AnyMatch(moved, p.φ, {T0}) then add a violation
9:      else if AnyMatch(D(X), p.φ, {T0, σ_X}) then add a violation
10:   for p ∈ Π(Y) of kind domain do
11:     if some insert or update in ops_X assigns a value outside p.φ then add a
      violation
12: return V
```

Static test. Overlap(φ_D, φ_R) is false only if both clauses are in $\mathcal{F}$ and $\varphi_D \wedge \varphi_R$ is unsatisfiable; it costs microseconds and reads no rows. If the clauses are disjoint, no row deleted under φ_D can satisfy φ_R with the values it had when deleted. The one remaining case is a row that X updated before deleting it: its value at the fork, which Y read, may still satisfy φ_R. Line 7 checks the fork values of exactly those rows.

Row check. When the static test cannot rule out an overlap, AnyMatch evaluates φ_R on each deleted row's value at the fork and at delete time. Both values come from the commit summaries; no data file is read. Clauses in $\mathcal{F}$ are evaluated directly, and any other single-table clause is evaluated by DuckDB [27] over those few rows. A row with no recorded value, or a clause that cannot be evaluated on its own, keeps the conflict. The row check costs $O(|D(X)|)$ per pair, so the static test is the fast path and the row check removes the false conflicts that the static test leaves, for example when the two clauses constrain different but correlated columns.

Other predicates. Lines 4–5 intersect live-row and foreign-key sets with $D(X)$; lines 10–11 test the partner's assigned values against `domain` clauses.

4.4 Row and Schema Reconciliation

The semantic and schema checks run first, so a blocked merge writes nothing. Rows are then reconciled under Criteria 1 and 2: one-sided and identical changes are adopted, updates on disjoint columns of one row are combined into one new version, and cell collisions become choices. Schemas are unified under Criterion 4; a dropped, renamed or narrowed column returns `schema_conflict`.

4.5 Physical Merge

For every row whose base version is replaced or removed, SemLake sets the row's position in its file's DV; the merged DV of a file is the union of both branches' DVs and the new positions. All new row versions from both branches go to one delta Parquet file. The merge then writes three small objects: the merge summary, a delta manifest that lists only the files whose DV or contents changed since T_0, and a manifest list that points to the base manifest and the delta manifest. Untouched file descriptors are inherited by reference. Base files are never rewritten; compaction is a separate, later step (Section 7.4).

4.6 A Merge Service for Many Writers

When many writers commit to one branch, each publish is a compare-and-swap on the head. A writer that loses the race does not abort: the service merges its commit onto the new head with the three-way merge. With *group commit*, the service takes up to G queued commits, merges them one by one onto the head, and publishes the batch with one compare-and-swap. A commit that conflicts with the head is returned to its writer. In this paper the service exists only inside the swarm simulation of Section 7.6.

5 CORRECTNESS AND COST

Assumption 1. *Every row deleted on a branch is deleted by a statement whose* `deletes_where` *clause the row satisfied at delete time, and the commit summaries record the deleted rows' values at delete time. The fork values are those of* T_0.

The assumption holds for SQL `DELETE` statements, whose clause the extractor records; a delete by row identifier must record a clause that names those identifiers.

Lemma 5.1 (Replay). *For fork b and tip t, Algorithm 1 folds every single-parent commit in* $\mathrm{anc}(t) \setminus \mathrm{anc}(b)$ *exactly once, in topological order, and no commit of* $\mathrm{anc}(b)$.

Proof. The walk stops at $\mathrm{anc}(b)$, so it visits exactly $\mathrm{anc}(t) \setminus \mathrm{anc}(b)$, and post-order emits a node after its parents. A parent inside $\mathrm{anc}(b)$ is mapped to $folded[b]$, so a commit whose parent predates b is still folded: if a report branch R forked from T_0 is merged with an update branch U into C_1, and C_1 is later merged with a purge branch forked from U, the new fork is U, and $R \in \mathrm{anc}(C_1) \setminus \mathrm{anc}(U)$ is folded with its parent mapped to U's delta. At a merge node u, the delta of the inner fork f is removed from both parents before they are combined, so commits shared by both parents are counted once. □

Theorem 5.2 (Soundness). *Under Assumption 1, if Algorithm 2 returns no violation for the deltas produced by Algorithm 1, the merge satisfies Criterion 3; in particular it is purge-safe.*

Proof. By Lemma 5.1 the deltas contain every deleted row and predicate since the fork. The live-row, foreign-key and domain parts follow from lines 4–5 and 10–11. For purge-safety, fix $\varphi_R \in \Pi(Y)$ and $r \in D(X)$. By Assumption 1, $\sigma_X(r) \models \varphi_D$ for some $\texttt{deletes_where}(\varphi_D) \in \Pi(X)$. If $\mathrm{Overlap}(\varphi_D, \varphi_R)$ is false, $\varphi_D \wedge \varphi_R$ is unsatisfiable, so $\sigma_X(r) \not\models \varphi_R$; if r is in *moved*, line 7 found $T_0.rows(r) \not\models \varphi_R$, and otherwise $T_0.rows(r) = \sigma_X(r)$. If Overlap is true, line 8 found that neither value satisfies φ_R. Either way Definition 3.3 holds for (r, φ_R). □

The check has false positives: the static test can report an overlap that the row check cannot clear when the read clause cannot be evaluated on its own (a join or subquery), and the definition itself counts a row that entered the read range only on X. Section 7.2 measures the false-positive rate. Theorem 5.2 is also checked by randomized tests: 200 random branch pairs, run directly and again after an intermediate merge, with both merge orders and against a brute-force oracle, find no missed conflict.

Cost. Let K be the number of single-parent commits since the fork on both sides. Replay issues K summary `GET`s. The check costs $O(|\Pi_A| \cdot |\Pi_B|)$ clause tests plus $O(|D|)$ row evaluations per overlapping pair, and reconciliation is linear in the changed rows. Publication writes one delta file and three metadata objects. None of these terms depends on the number of files or rows in the table.

Read overhead. Let a table have G row groups, G_d of them with a non-empty DV. A scan reads clean row groups at base cost and pays a masking cost on dirty ones, so its overhead grows with G_d/G. Localized changes keep G_d/G small; deletes spread across the table make every row group dirty until compaction. This is a cost model, which Section 7.4 measures.

6 IMPLEMENTATION

The prototype is about 2,100 lines of Python in six modules (metadata, storage, merge, verification, SQL extraction and the baselines), plus about 4,700 lines of benchmark code. Data files are Parquet written with `pyarrow`, with a hidden `__row_id` column for row lineage. DVs are Roaring bitmaps (`pyroaring`) serialized into the file descriptors. Snapshot row maps are copy-on-write overlays that store only each commit's changes. Each snapshot caches its ancestor generations, and the LCA search stops once the highest open generation falls below the best common ancestor found.

The prototype has three simplifications that the evaluation inherits.

- *Object store.* Objects live in an in-memory store that counts `PUT`, `GET` and `LIST` requests and bytes. Request costs use S3 Standard prices [3], and end-to-end latency is estimated as measured CPU time plus 100 ms per `PUT`, 30 ms per `GET`, 50 ms per `LIST` and 100 MB/s of transfer, round-trip times in the range Durner et al. report [13]. No experiment runs against S3.
- *Scans.* The read experiments store a DV as a Parquet mask of booleans and apply it in DuckDB with a positional join; clean row groups are read without a mask. A native reader would apply the Roaring bitmap directly.
- *Service functions.* Compaction and the group-commit merge service are implemented in the benchmark harness, not in the engine.

B1, B1R, B2 and B6 are rule models evaluated on the same snapshots as SemLake: they compute what each system's documented rule decides, not what an engine would do. B3 and B4 are our reimplementations. B5 is the Dolt binary [12], driven through its SQL interface with `dolt merge`.

Table 3: Conflict detection on (a) the labelled scenarios and (b) the TPC-H refresh-versus-report pairs. TP and FP count reported semantic conflicts; Ab. counts write-conflict aborts, which block a merge without naming a semantic reason.

	(a) 27 scenarios, 14 conflicts					(b) 42 TPC-H pairs, 22 conflicts			
Rule	TP	FP	Ab.	Rec.	Prec.	TP	FP	Rec.	Prec.
B1 file OCC	0	0	1	0.00	–	0	0	0.00	–
B1R Delta-style	11	6	0	0.79	0.65	22	3	1.00	0.88
B2 copy-on-write	0	0	0	0.00	–	0	0	0.00	–
B3 tuple versioning	0	0	0	0.00	–	0	0	0.00	–
B4 CRDT	0	0	0	0.00	–	0	0	0.00	–
B5 Dolt	2	0	0	0.14	1.00	0	0	0.00	–
B6 SSI	0	0	0	0.00	–	0	0	0.00	–
SemLake, static test only	14	2	0	1.00	0.88	22	8	1.00	0.73
SemLake	14	0	0	1.00	1.00	22	0	1.00	1.00

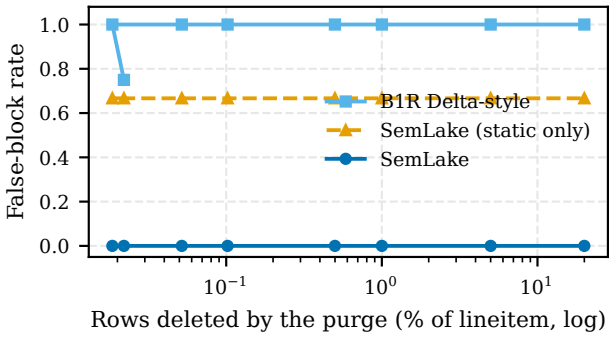


Figure 2: False-block rate when a purge that removes no row read by Q6 runs next to a Q6 report, against the fraction of `lineitem` the purge deletes. Each point is 12 purges in three clause forms (conjunctive, `NOT` and `OR`).

7 EVALUATION

We ask four questions. **Q1:** does SemLake catch read–delete hazards without blocking safe merges (M1)? **Q2:** how do merge time and requests grow with contention, table size and history (M2, M3)? **Q3:** what do open DVs cost readers, and when should they be compacted (M3, M4)? **Q4:** what does the merge service give a swarm of concurrent writers?

7.1 Setup

All experiments run on one machine with 4 cores, Python 3.13.16, DuckDB 1.5.6 and pyarrow 26.0.0. TPC-H data is real `lineitem` generated by `tpchgen-cli` [29], a port of the TPC-H generator [30]. Timed merges run once untimed and then 5 times, and we report the median with the range; DuckDB scans run in separate processes one after another; the swarm runs 5 seeds and reports means with 95% confidence intervals.

7.2 Correctness (Q1)

Labelled scenarios. The suite has 27 merge scenarios, 14 of which must conflict. They cover shared-file updates, equality, range, `BETWEEN`, `IN` and multi-column clauses, boundary cases, clauses with `NOT` and `OR`, domain rules, foreign keys, live-row predicates and additive schema changes. Table 3(a) gives the results. SemLake finds all 14 conflicts with 0 false positives. The physical rules B1, B2, B3 and B4 never report a semantic conflict. Dolt catches the declared foreign-key and `CHECK` cases (recall 0.14): a report's filter cannot be declared as a constraint, and Dolt refuses to add a `CHECK` constraint that legacy rows violate. B1R finds the read–delete cases whose rows share a file with the deleted rows (recall 0.79) but blocks 6 safe merges. B6 reports 0 conflicts: the suite's hazards are one-way dependencies, which SSI accepts, as Proposition 3.4 predicts.

TPC-H pairs. We generate `lineitem` at scale factor 0.01 (60,175 rows), sort it by `l_shipdate`, and write it in 31 files. Six refresh branches (four ship-date purges and two RF2-style deletes of order keys) meet seven report branches that read with the `lineitem` filters of Q1, Q3, Q6, Q7, Q10, Q12 and Q14, for 42 pairs. A pair is a true conflict if a deleted row satisfies the report's filter; 22 pairs are. Table 3(b) shows that SemLake finds all of them with precision 1.00; with the row check disabled, precision is 0.73. B1R also finds all of them but blocks 3 safe pairs (precision 0.88). Every other rule, including Dolt, reports none.

Selectivity. Figure 2 isolates false blocks. A report reads with Q6's filter, and a purge deletes a random range of part keys restricted to discounts Q6 does not read, so no pair is a true conflict. The purge removes from 0.019% to 20% of the table. B1R blocks the report as soon as the purge touches one of the 6 files Q6 reads; its false-block rate is 1.00 at 0.1% of rows. The static test alone blocks the `NOT` and `OR` forms, a rate of 0.67. With the row check, SemLake blocks none.

7.3 Merge Latency and Requests (Q2)

Figure 3 shows merge CPU time.

- *Contention (W2).* On a 10K-file table, SemLake merges in 9.02 ms at $\kappa = 0$ and 8.12 ms at $\kappa = 20\%$, where colliding cells become choices; B1 takes 227.3 ms and 203.2 ms because it compares full file lists. B4 takes 37.9 ms at $\kappa = 0$, but it checks nothing: it overwrites one side of every collision.
- *Table size (W4).* As the manifest grows from 1K to 10K files, SemLake's merge of a schema change and an append takes 0.38 ms and 3.22 ms. B1, B2 and B3 take 147.0 ms, 129.8 ms and 171.4 ms at 10K files; B3 issues 20,009 requests to rebuild its dictionaries.
- *History (W4).* From 42 to 360 snapshots, the LCA search takes 3.5 and 6.6 μs and the merge 0.53 ms and 1.20 ms. Each merge here replays one commit per side, so replay does not explain the growth; the merge also lists the manifest prefix, which grows with the history in our store, and we did not isolate the cause further.

CPU time is not the whole story. With the latency model of Section 6, a W2 merge at $\kappa = 0$ is estimated at 0.55 s for SemLake and 0.48 s for B1, because SemLake reads one summary per commit and writes more metadata objects. What SemLake avoids is growth with table size: on W1 with 10K files, where each branch updates rows in half of the files, SemLake issues 7 requests (3 `PUT`, 3 `GET`, 1 `LIST`) and writes 380 bytes, while B2 and B3 issue 20,007 requests and are estimated at 1,301 s. B1 issues only 5 requests, but it aborts this merge.

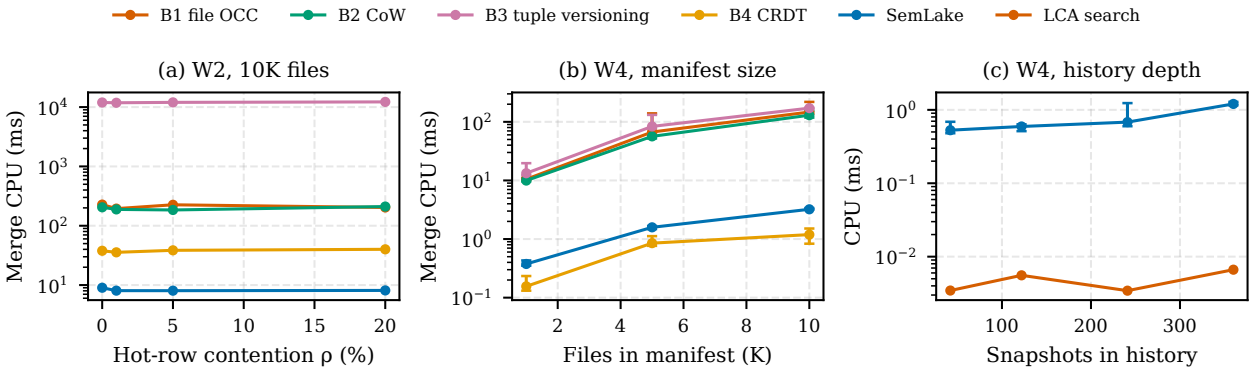


Figure 3: Merge CPU time (median of 5 runs; bars show the range) as (a) hot-row contention grows on a 10K-file table, (b) the manifest grows, and (c) the commit history grows.

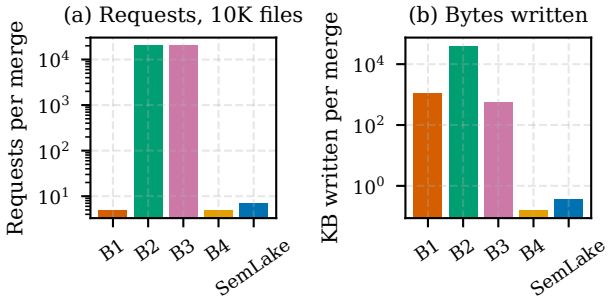


Figure 4: Object-store requests and bytes written per W1 merge on a 10K-file table.

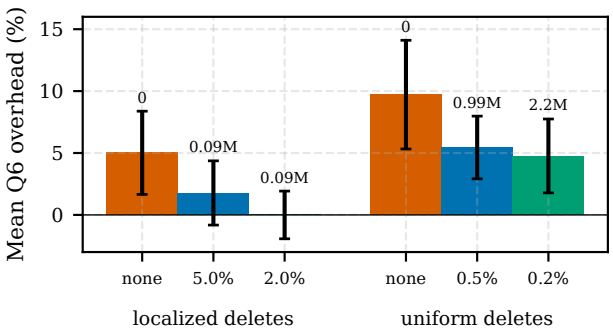


Figure 5: Mean Q6 overhead over 40 merges, each deleting 0.1% of the rows of SF 1 `lineitem`, by compaction threshold (fraction of a row group's rows with pending deletes). Bars show 95% intervals over the merges; the label above each bar is the number of rows rewritten per merge.

Table 4: TPC-H Q6 on SF 1 `lineitem` after a merge left deletes in a DV: median time of 5 processes, and overhead with a 95% confidence interval.

Deletes	Rows	Clean groups	Base (ms)	DV (ms)	Overhead (95% CI)
localized	1%	60/61	190.5	198.2	+0.9% (−3.4 to +6.9%)
uniform	0.1%	0/61	194.0	206.8	+4.2% (−1.7 to +12.5%)
uniform	1%	0/61	187.4	207.2	+6.0% (−5.3 to +23.1%)
uniform	5%	0/61	178.6	187.8	+5.4% (+0.2 to +14.2%)

7.4 Read Overhead and Compaction (Q3)

Table 4 runs TPC-H Q6 in DuckDB (one thread) over real SF 1 `lineitem` (6,100,000 rows in 61 row groups of 100,000 rows) after a merge left a DV open. When 1% of the rows are deleted inside one row group, the overhead is +0.9% (95% CI −3.4% to +6.9%). When the same deletes are spread over every row group, every group needs the mask, and the overhead is +6.0%. Both intervals are wide relative to the medians: on one thread over SF 1, the masking cost is small next to the scan itself. The mask is applied by a positional join in our implementation; we did not measure a native DV reader.

Figure 5 follows a table through 40 merges, each deleting 0.1% of its rows, and measures Q6 after every merge. A policy rewrites a row group once the rows deleted in it but still present in its file pass a threshold. Without compaction, the mean overhead is 5.0% for localized deletes and 9.7% for uniform ones. With localized deletes, a 2% threshold brings it to 0.0% and rewrites 89,549 rows per merge, almost one row group. With uniform deletes, the best threshold we tried (0.2%) gives 4.8% but rewrites 2,233,892 rows per merge, over a third of the table, because every row group crosses the threshold at about the same time. Per-merge timings vary by more than these differences, so only the means are meaningful. Compaction therefore pays off for localized deletes; for uniform deletes it costs nearly a copy-on-write rewrite to remove a few percent of scan time.

Figure 6 combines the measured requests, bytes and overheads into dollars. At 10 scans per merge, 1,000 merges cost $0.086 with localized DVs and $0.088 with uniform ones, against $0.228 for copy-on-write with 128 MiB files. Leaving DVs open stays cheaper than rewriting until 3,094 scans per merge for localized deletes and

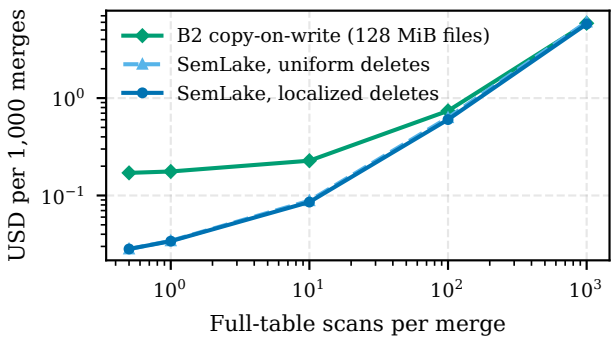


Figure 6: Dollars per 1,000 merges (requests, one day of storage for written bytes, and CPU at $0.10/hour) against the number of full scans per merge, using the overheads of Table 4.

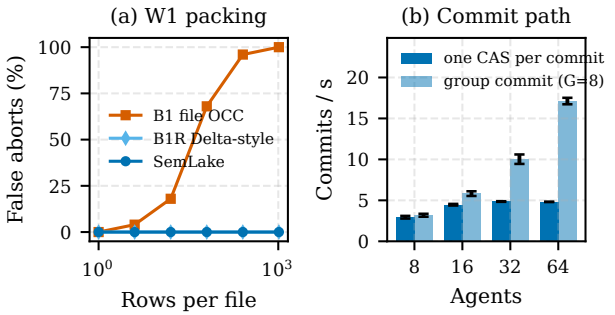


Figure 7: (a) False aborts on W1 as rows per file grow. (b) Swarm commit throughput with one compare-and-swap per commit and with group commit ($G = 8$).

446 for uniform ones. Past that point the merged files should be compacted.

7.5 Ablations

File packing. Two branches each update 4 random rows of a 1,024-row table. As rows per file grow from 1 to 1,024, B1's false-abort rate rises from 0 to 1.00 (Figure 7a). B1R and SemLake never abort, because both compare rows.

Row check. Disabling the row check leaves recall unchanged and lowers precision from 1.00 to 0.88 on the labelled suite and from 1.00 to 0.73 on the TPC-H pairs; Figure 2 shows the false blocks it removes.

Replay. Reading only the tips' summaries loses the predicates of commits merged earlier. The edge-case tests include a report that was merged once before it meets a purge; tip-only replay merges that pair, and Algorithm 1 blocks it.

Group commit. With one compare-and-swap per commit, the swarm of Section 7.6 reaches 4.8 commits/s at 64 agents; with group commit it reaches 17.1 (Figure 7b).

Table 5: Simulated swarm, 60 s per run, mean of 5 seeds (± 95% CI). Haz.: hazards committed and caught, summed over seeds. Blocks and redos: false semantic blocks and false file-conflict redos per run.

N	Rule	Commits/s	Haz.	Caught	Blocks	Redos
8	B1 file OCC	1.90 ± 0.21	2	0	0.0	91.4
8	B1R Delta-style	2.92 ± 0.19	2	2	4.8	0.0
8	B2 copy-on-write	0.65 ± 0.06	0	0	0.0	0.0
8	SemLake	2.94 ± 0.16	0	4	0.0	0.0
8	SemLake, group commit	3.18 ± 0.18	0	4	0.0	0.0
32	B1 file OCC	4.09 ± 0.11	5	0	0.0	488.8
32	B1R Delta-style	5.00 ± 0.04	4	9	19.6	0.0
32	B2 copy-on-write	1.40 ± 0.15	0	0	0.0	0.0
32	SemLake	4.86 ± 0.04	0	12	0.2	0.0
32	SemLake, group commit	10.02 ± 0.58	0	19	0.2	0.0
64	B1 file OCC	5.02 ± 0.05	20	0	0.0	895.6
64	B1R Delta-style	5.13 ± 0.01	11	20	49.6	0.0
64	B2 copy-on-write	1.92 ± 0.25	0	0	0.0	0.0
64	SemLake	4.81 ± 0.01	0	43	0.4	0.0
64	SemLake, group commit	17.12 ± 0.40	0	59	1.2	0.0

7.6 Agent Swarm (Q4)

Agent frameworks branch a shared table per agent and merge the results back [17, 34]. We simulate this. N agents repeatedly branch from `main`, work for a random time (mean 2 s), and commit one task: an update of four rows (80%), a range delete (5%) or a report that reads a range and writes an aggregate (15%). The table has 32 files of 32 rows. Publishing is a compare-and-swap with a 10 ms catalog round trip. The simulation is discrete-event: SemLake runs on the real engine, and each merge advances simulated time by its measured CPU time plus its counted requests under the latency model; the baselines are rule models charged with the same latency model. A report and a delete that ran concurrently over shared rows are a hazard if both commit. Each run lasts 60 s of simulated time. We also run a purge-heavy mix (15% deletes, 30% reports) at 8 and 32 agents; hazard totals below cover both mixes.

Table 5 and Figure 8 give the results. B1 redoes work on every file overlap and B2 rewrites every touched file. B1R and SemLake both avoid file conflicts, but B1R blocks a report whenever a concurrent delete touched a file it read: 19.6 blocks per run at 32 agents, against 0.2 for SemLake. Over all runs, B1 committed 63 hazards, B2 9, B1R 27, and SemLake 0. B1R still commits hazards because its read check is one-directional: it rejects a report that commits after a delete of rows in files it read, but a delete that commits after the report read nothing the report wrote, so it goes through. With one compare-and-swap per commit, SemLake's throughput is limited by the merge's object-store round trips (4.8 commits/s at 64 agents); group commit raises it to 17.1.

8 RELATED WORK

Lakehouse formats and catalogs. Delta Lake [6], Iceberg [4] and Hudi [19] add transactions to object-store tables, and their concurrency control compares written files or, in Delta's row-level mode, written rows and read files [22]. Deletion vectors and row lineage make row-level changes cheap [21, 25]. Comparisons of these formats [10, 19] study single-branch workloads. Multi-table transactions [16] and catalog engines [24] coordinate commits across tables

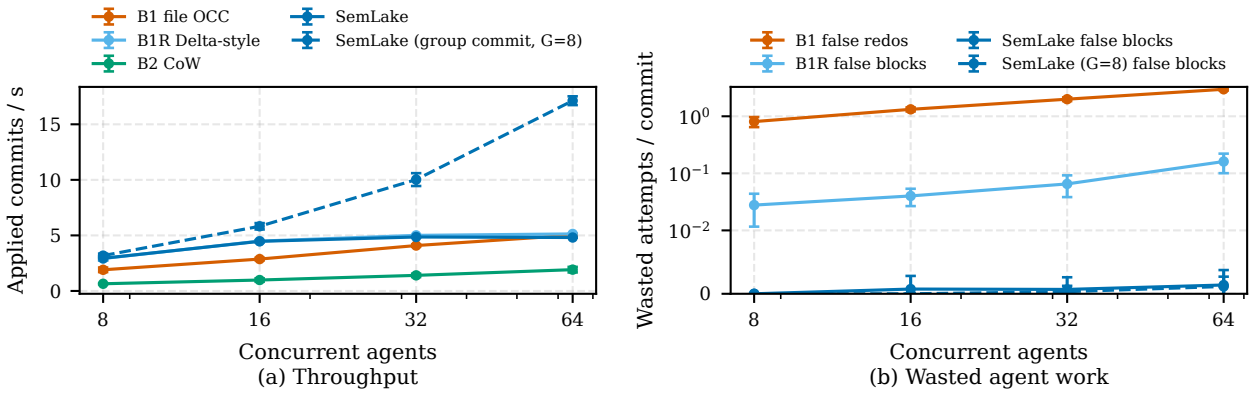


Figure 8: Simulated swarm: (a) applied commits per second and (b) wasted attempts per applied commit (false redos for B1, false blocks for B1R and SemLake), mean of 5 seeds with 95% confidence intervals.

but do not merge branches. lakeFS [31] and Nessie [26] provide Git-like branches over lakehouse tables and merge them by comparing which objects or tables changed. SemLake is complementary: it decides whether two branches of one table can be merged, at row and predicate granularity.

Dataset versioning. DataHub [7], Decibel [23], OrpheusDB [18], ForkBase [32] and the storage trade-offs of Bhattacherjee et al. [8] focus on storing and retrieving versions. Dolt [12] merges cells and checks declared constraints, which we compare against.

Concurrency control. Predicate locks [14] block a write that falls in a range another transaction read, which is the online counterpart of our static test; SemLake applies the test after the fact, to two finished branches, and adds a check of the deleted rows' values. Adya's dependency graphs [1] and SSI [9] define and enforce serializability; Proposition 3.4 shows why a published merge needs a different invariant. CRDTs [2, 15] always converge, which on W2 means losing one write.

Program merge. Verified three-way merge [28], version-space representations of conflicts [35] and semantic-conflict detection by tests [11] treat merge as a semantic question over programs. SemLake asks the same question over table predicates and deleted rows.

Agents on shared data. Git4Data [17] and Chronos [34] give agents branches over databases. Git4Data names cross-branch semantic conflicts as open; SemLake is one answer for lakehouse tables.

9 LIMITATIONS AND CONCLUSION

Limitations. The prototype runs in one Python process over an in-memory object store; request costs and latencies are modelled, not measured on S3. Scans apply DVs through a Parquet mask and a positional join rather than a native bitmap reader. Compaction and the merge service live in the benchmark harness, compaction was measured on one thread at SF 1 only, and the swarm is a simulation. B1, B1R, B2 and B6 are rule models of the systems' documented behaviour. The TPC-H pairs use scale factor 0.01 and the read experiments scale factor 1; larger scales were not run. Purge-safety covers deletes, not updates that move rows out of a read range, and a read clause with a join or subquery cannot be checked against rows and is treated as overlapping. Schema widening follows the prototype's list, which includes the lossy integer-to-double case.

Conclusion. Branch merges in lakehouses are decided today by which files or rows two branches wrote, which both aborts safe merges and accepts unsafe ones. SemLake decides them from what the branches read and deleted, using summaries that each commit already has to write. A static clause test handles most pairs at once, a check of the deleted rows' values removes the remaining false conflicts, and the merge publishes deletion vectors instead of rewriting files. In our experiments this caught every read–delete hazard, blocked far fewer safe merges than file-granularity read checks, and kept merge requests independent of table size.

REFERENCES

[1] Atul Adya. 1999. *Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions.* Ph.D. Dissertation (Tech. Rep. MIT-LCS-TR-786). Massachusetts Institute of Technology.

[2] Paulo Sérgio Almeida. 2024. Approaches to Conflict-free Replicated Data Types. *ACM Comput. Surv.* 56, 8 (2024), 202:1–202:36.

[3] Amazon Web Services. 2026. *Amazon S3 Pricing: S3 Standard, US East (N. Virginia).* Price List. Amazon Web Services, Inc. https://aws.amazon.com/s3/pricing/.

[4] Apache Software Foundation. 2025. *Apache Iceberg Table Format Specification: Version 3 (Deletion Vectors and Row Lineage).* Technical Specification Format Version 3. The Apache Software Foundation. https://iceberg.apache.org/spec/.

[5] Apache Software Foundation. 2026. *Apache Iceberg Table Configuration: Write Properties.* Technical Documentation. The Apache Software Foundation. https://iceberg.apache.org/docs/latest/configuration/.

[6] Michael Armbrust, Tathagata Das, Liwen Torres, Burak Yavuz, Shixiong Zhu, Reynold Xin, Ali Ghodsi, Ion Stoica, and Matei Zaharia. 2020. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proc. VLDB Endow.* 13, 12 (2020), 3411–3424.

[7] Anant Bhardwaj, Souvik Bhattacherjee, Amit Chavan, Amol Deshpande, Aaron J. Elmore, Samuel Madden, and Aditya Parameswaran. 2015. DataHub: Collaborative Data Science and Dataset Version Management at Scale. In *Proc. Conf. on Innovative Data Systems Research (CIDR).*

[8] Souvik Bhattacherjee, Amit Chavan, Silu Huang, Amol Deshpande, and Aditya Parameswaran. 2015. Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff. *Proc. VLDB Endow.* 8, 12 (2015), 1346–1357.

[9] Michael J. Cahill, Uwe Röhm, and Alan D. Fekete. 2009. Serializable Isolation for Snapshot Databases. *ACM Trans. Database Syst.* 34, 4 (2009), 20:1–20:42.

[10] Jesús Camacho-Rodríguez, Ashvin Agrawal, Anja Gruenheid, Ashit Gosalia, Cristian Petculescu, Josep Aguilar-Saborit, Avrilia Floratou, Carlo Curino, and Raghu Ramakrishnan. 2024. LST-Bench: Benchmarking Log-Structured Tables in the Cloud. *Proc. ACM Manag. Data* 2, 1 (2024), 56:1–56:26.

[11] Leuson da Silva, Paulo Borba, Arthur Toni, and Rodrigo Bonifácio. 2024. Detecting Semantic Conflicts with Unit Tests. *Journal of Systems and Software* 212 (2024), 112006.

[12] DoltHub, Inc. 2026. *Dolt: Git for Data (Version-Controlled SQL Database).* Software Repository Version 2.5.0, built from source. DoltHub, Inc. https://github.com/dolthub/dolt.

[13] Dominik Durner, Viktor Leis, and Thomas Neumann. 2023. Exploiting Cloud Object Storage for High-Performance Analytics. *Proc. VLDB Endow.* 16, 11 (2023), 2769–2782.

[14] Kapali P. Eswaran, Jim N. Gray, Raymond A. Lorie, and Irving L. Traiger. 1976. The Notions of Consistency and Predicate Locks in a Database System. *Commun. ACM* 19, 11 (1976), 624–633.

[15] Nuno Faria and José Pereira. 2025. CRDV: Conflict-free Replicated Data Views. *Proc. ACM Manag. Data* 3, 1 (2025), 42:1–42:25.

[16] Tobias Götz, Daniel Ritter, Muhammad El-Hindi, and Jana Giceva. 2026. Interoperable ACID Transactions for Open Table Formats. *Proc. VLDB Endow.* 19, 11 (2026), 3216–3230.

[17] Hongshen Gou, Zuyu Zhang, Yuze Sun, Peng Xu, Feng Tian, Long Wang, and Jianguo Wang. 2026. Git4Data: Database-Native Version Control for AI Agents. *arXiv preprint arXiv:2609.02106* (2026).

[18] Silu Huang, Liqi Xu, Mengxiao Xia, and Aditya Parameswaran. 2017. OrpheusDB: Bolt-on Versioning for Relational Databases. *Proc. VLDB Endow.* 10, 10 (2017), 1130–1141.

[19] Paras Jain, Peter Kraft, Conor Power, Tathagata Das, Ion Stoica, and Matei Zaharia. 2023. Analyzing and Comparing Lakehouse Storage Systems. In *Proc. Conf. on Innovative Data Systems Research (CIDR).*

[20] Daniel Lemire, Owen Kaser, Nathan Kurz, Luca Deri, Chris O'Hara, François Saint-Jacques, and Gregory Ssi-Yan-Kai. 2018. Roaring Bitmaps: Implementation of an Optimized Software Library. *Softw. Pract. Exp.* 48, 4 (2018), 867–895.

[21] Linux Foundation Delta Lake Project. 2024. *Delta Transaction Log Protocol: Deletion Vectors and Row Tracking Specification.* Protocol Specification Reader v3 / Writer v7. The Linux Foundation. https://github.com/delta-io/delta/blob/master/PROTOCOL.md.

[22] Linux Foundation Delta Lake Project. 2026. *Delta Lake Documentation: Concurrency Control.* Technical Documentation. The Linux Foundation. https://docs.delta.io/latest/concurrency-control.html.

[23] Michael Maddox, David Goehring, Aaron J. Elmore, Samuel Madden, Aditya Parameswaran, and Amol Deshpande. 2016. Decibel: The Relational Dataset Branching System. *Proc. VLDB Endow.* 9, 9 (2016), 624–635.

[24] Seongjin Oh, Hoang Nguyen, and Yanlei Diao. 2025. TreeCat: Standalone Catalog Engine for Large Data Systems. *Proc. VLDB Endow.* 18, 11 (2025), 3845–3858.

[25] Anton Okolnychyi, Ryan Blue, Russell Spitzer, Jack Chen, and John Wang. 2024. Petabyte-Scale Row-Level Operations in Data Lakehouses. *Proc. VLDB Endow.* 17, 12 (2024), 4159–4171.

[26] Project Nessie. 2026. *Nessie: Transactional Catalog for Data Lakes with Git-like Semantics.* Software Repository. Project Nessie. https://github.com/projectnessie/nessie.

[27] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19).* ACM, Amsterdam, The Netherlands, 1981–1984.

[28] Marcelo Sousa, Isil Dillig, and Shuvendu K. Lahiri. 2018. Verified Three-Way Program Merge. *Proc. ACM Program. Lang.* 2, OOPSLA (2018), 165:1–165:29.

[29] tpchgen-rs Contributors. 2026. *tpchgen-rs: TPC-H Data Generator in Rust (tpchgen-cli).* Software Repository tpchgen-cli 3.0.0. tpchgen-rs Project. https://github.com/clflushopt/tpchgen-rs.

[30] Transaction Processing Performance Council. 2022. *TPC Benchmark H (TPC-H) Standard Specification.* Standard Specification Revision 3.0.1. Transaction Processing Performance Council (TPC). http://www.tpc.org/tpch/.

[31] Treeverse. 2026. *lakeFS: Data Version Control for Data Lakes.* Software Repository. Treeverse. https://github.com/treeverse/lakeFS.

[32] Sheng Wang, Tien Tuan Anh Dinh, Qian Lin, Zhongle Xie, Meihui Chang, Gang Chen, and Beng Chin Ooi. 2018. ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications. *Proc. VLDB Endow.* 11, 10 (2018), 1137–1150.

[33] Matei Zaharia, Ali Ghodsi, Reynold Xin, and Michael Armbrust. 2021. Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *Proc. Conf. on Innovative Data Systems Research (CIDR).*

[34] Xinjing Zhou, Jason Mohoney, Samuel Madden, Michael Stonebraker, and Lei Cao. 2026. Chronos: Efficient Bolt-On Branching Across Data Stores for Stateful Agentic Applications. *arXiv preprint arXiv:2609.14889* (2026).

[35] He Zhu and Fei He. 2018. Conflict Resolution for Structured Merge via Version Space Algebra. *Proc. ACM Program. Lang.* 2, OOPSLA (2018), 166:1–166:28.